

Part 12 — Static Analysis vs Human Review

1. What did Pylint detect well?

Pylint was very good at catching **mechanical, rule-based problems**, things you can check just by looking at the code's shape, without understanding what it's supposed to do:

- Style issues: trailing whitespace, lines too long, bad indentation.
- Naming rules: `df`, `e`, `T` not matching the expected naming style.
- Unused stuff: unused imports, unused variables (like `workbook` in `export_utils.py`).
- Import problems: imports that fail, imports placed inside functions instead of at the top.
- Size limits: too many arguments, too many local variables, too many statements in one method (this is how it caught `register_user` being too long).
- Basic exception-handling mistakes: catching a bare `except:` with no exception type, or catching `Exception` too broadly, or re-raising without `raise ... from e`.
- Copy-pasted code — but only when it's copied **across different files**, not within the same file (more on this below).

2. What needed a human to catch?

Several of the most serious problems in the project were things Pylint could never see, because they're about **whether the code is correct or well-designed**, not about its shape:

- **A broken password-reset flow.** The reset token never actually stored the user's email, so the reset feature could never work. The code looked fine and even had docstrings, a linter has no way to know the logic doesn't connect.
- **A wrong calculation from a magic number.** `submission_rate = total_submissions / 25` assumed every class has exactly 25 students. This gives wrong numbers for any other class size, but `25` is just an ordinary number to Pylint, nothing about it looks wrong.
- **Duplicated logic within the same file.** `export_service.py` repeated the same ~25-line export-and-manifest block three times, but with different variable names each time. Pylint's duplicate-code checker compares lines fairly literally, so different names made it invisible.
- **Two conflicting systems for the same idea.** The `Student` class kept its own private `_grades` dictionary, completely separate from the "real" `Grade/GradeService` system used everywhere else. Both pieces of code were fine on their own — the problem only exists when you compare them and notice grades recorded one way never show up the other way. That kind of cross-file, "do these two things actually agree with each other" reasoning is something only a human reading both files can do.

- Debug `print()` statements used as logging, some of them printing personal information (name, email, phone, address). Pylint has no rule against `print()` at all, and doesn't think about privacy.

3. Did Pylint report anything not worth fixing?

Yes, a few examples:

- **R0801 duplicate-code** on the imports in `user_repository.py` and `auth_service.py`. Both files import the same five model classes (`User`, `Student`, `Teacher`, `Parent`, `Admin`). This isn't real duplication of logic, it's just two files needing the same imports because they both work with all user types. "Fixing" it would mean something artificial like re-exporting from a shared module for no real benefit. I'd treat this as a false positive.
- **fail-under=10.0** isn't a code issue exactly, but it's an unrealistic default setting: requiring a perfect 10/10 score just to pass would block basically every commit on a real project. Not wrong, just not a useful bar to hold the team to.
- Some of the **naming complaints** (like flagging `df` for a pandas DataFrame) are technically correct but not worth acting on, since `df` is such a standard, well-understood convention in that ecosystem that renaming it would only hurt readability.

4. Can a program score high on Pylint and still be badly designed?

Yes, and this project proves it. A file can be clean by Pylint's rules — good naming, short methods, no unused imports, while still being badly designed underneath. For example:

- `export_service.py` could, in principle, get individual methods that each look small and tidy, while still having the *same* block of logic pasted three times across the file. Pylint sees three well-formed methods; a human sees one idea that should be one method.
- The `Student` model's private grade-tracking and the real `GradeService` system could both individually pass every Pylint check — sensible names, reasonable method lengths, docstrings present, while being a serious design flaw *together*, because they quietly disagree about where grades live.
- The broken password-reset flow read perfectly fine line-by-line (good names, a docstring, no unused variables) while being completely non-functional.

5. Should static analysis replace human review, complement it, or only be used occasionally?

Complement it not replace it, and not just occasionally either. It should run on every change, but as a first pass, not the final word.

Reasoning:

- Static analysis is fast, consistent, and never gets tired, perfect for catching the mechanical stuff (naming, unused imports, overly long methods, obviously bad exception handling) on every single commit. Doing this by hand every time would be a waste of a person's attention.
- But as shown above, static analysis has no idea whether the code is *correct*, whether two parts of the system *agree* with each other, or whether a number like 25 represents a real assumption that will break. Only a human who understands the feature can catch that.
- Using it "only occasionally" would be worse than not having it at all in a way, problems would pile up between check-ins, making them harder to trace and fix.